

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: Ch. Gouri Sai Sree

Roll No.: A24126552076

Date: 1-09-2026

1. TITLE

Develop Dynamic Webpage Using Java Script

2. AIM

- To develop a dynamic, interactive webpage using **HTML, CSS, and JavaScript**. ● To create a **Dynamic Student Dashboard** that includes a live digital clock, theme toggle functionality, a percentage calculator, a motivational quote generator, and a student registration form with validation.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of developing dynamic, interactive webpages using JavaScript.
- To use JavaScript functions and methods to perform real-time operations like clock updates.
- To dynamically manipulate the DOM to update page content based on user interaction.
- To implement input validation for the percentage calculator and registration form.
- To toggle interface themes dynamically using CSS classes.
- To handle event listeners to trigger dashboard features like quote generation and theme switching.

4. THEORY

JavaScript is a programming language used to make webpages dynamic and interactive, allowing content to be created, modified, and removed based on user actions.

In this application, JavaScript is used to develop a **Dynamic Student Dashboard** that helps users manage academic tasks.

- **HTML** provides the structure for the dashboard components (clock, calculator, form,

quotes).

- **CSS** provides the styling, including transitions for dark mode.
- **JavaScript functions** manage the logic:
 - `Date()` and `setInterval()` maintain the live clock.
 - `Math.random()` handles the dynamic quote generation.
 - `classList.toggle()` manages the theme switching.
 - **Event listeners** (`addEventListener`) handle form submissions and button interactions.
 - **DOM methods** (`getElementById`, `innerHTML`) access and update elements based on user input.

The basic flow of the application is:

User Interaction → JavaScript Logic (Logic/Validation/Math) → DOM Update (Dynamic UI Changes)

5. ALGORITHM / PROCEDURE

1. Create the HTML structure containing a header, live clock display, theme toggle button, student info section, percentage calculator, motivational quote display, and registration form.
2. Implement dark mode theme toggle functionality using `classList.toggle()` attached to the theme button event listener.
3. Create a digital clock function utilizing `Date()` and `setInterval()` to dynamically update and display the current time every second.
4. Implement a percentage calculator that validates input marks to ensure they fall within the range of 0 to 100, then computes and displays the average percentage.
5. Create a quote generator function that selects and displays random motivational quotes from a predefined array upon button click.
6. Implement form validation on submission that checks for empty required fields and validates email format before confirming registration.
7. Apply CSS rules to layout and style page components, and define properties for dark mode styling.

6. PROGRAM

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Color Changer</title>

  <style>
    body {
      text-align: center;
      font-family: Arial, sans-serif;
      padding-top: 100px;
      background-color: lightblue;
      transition: 0.5s;
    }
  </style>
</head>
<body>
```

```

button {
  padding: 12px 20px;
  margin: 5px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
}

#changeBtn {
  background-color: darkblue;
  color: white;
}

#resetBtn {
  background-color: gray;
  color: white;
}
</style>
</head>

<body>

<h1 id="title">Dynamic Color Page</h1>
<p id="message">Click "Change Color" to change the background.</p>

<button id="changeBtn" onclick="changeColor()">Change Color</button>
<button id="resetBtn" onclick="resetColor()">Reset</button>

<script>
  let colors = ["lightblue", "lightgreen", "lightcoral", "lightyellow", "plum"];
  let currentColor = 0;

  function changeColor() {
    currentColor++;

    if (currentColor >= colors.length) {
      currentColor = 0;
    }

    document.body.style.backgroundColor = colors[currentColor];

    document.getElementById("message").innerHTML =
      "Current color: " + colors[currentColor];
  }

  function resetColor() {
    currentColor = 0;

    document.body.style.backgroundColor = colors[currentColor];

    document.getElementById("message").innerHTML =
      "Color has been reset!";
  }
</script>

```

</body>
</html>

7. CODE EXPLANATION

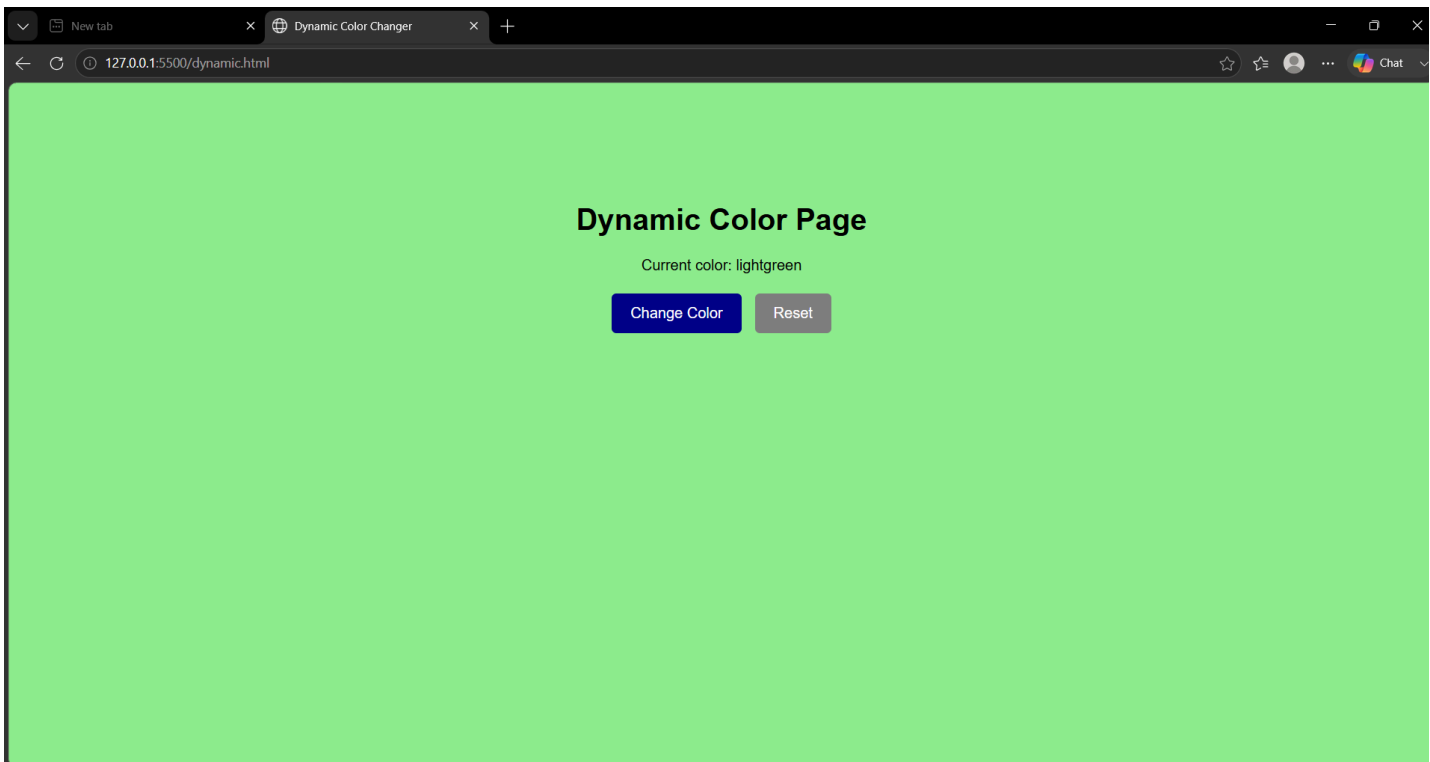
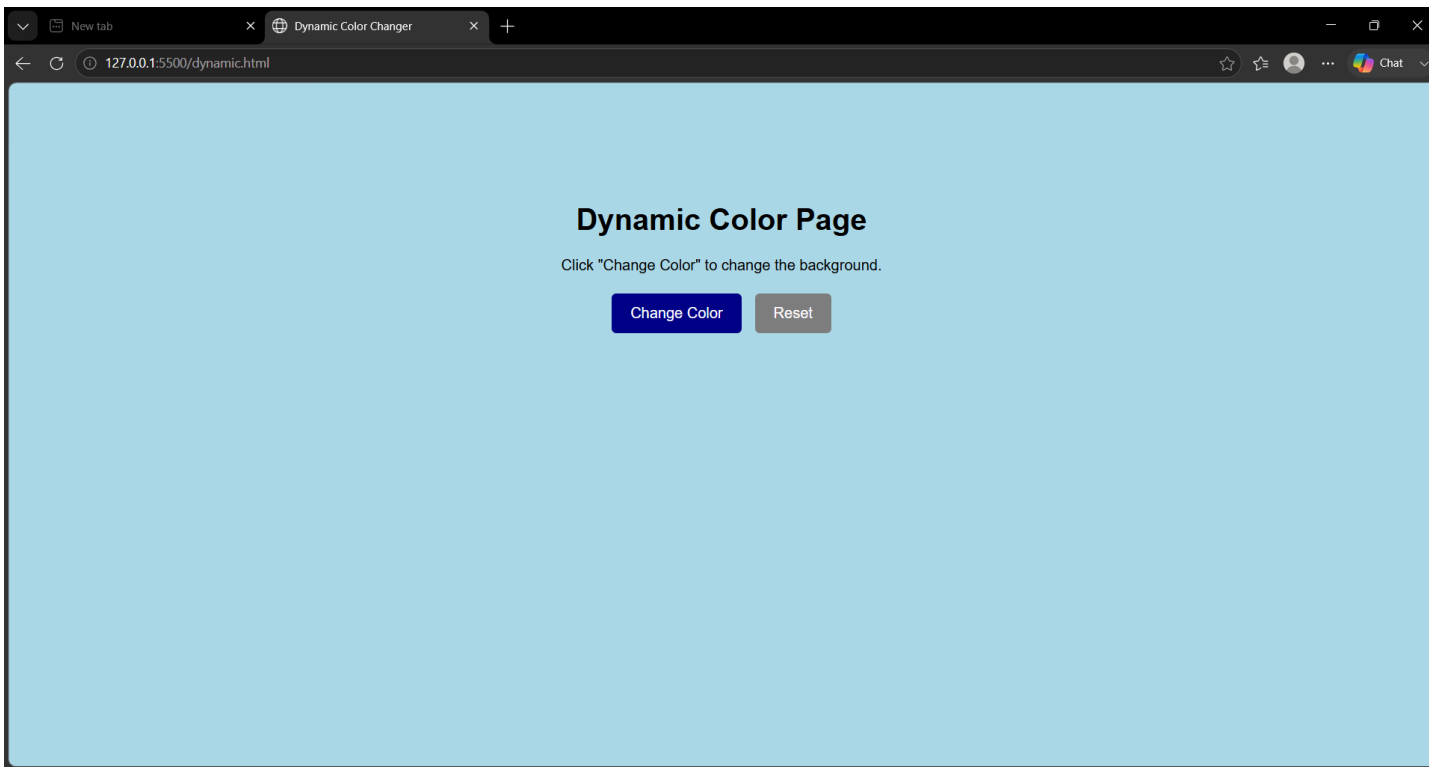
Component	Description
themeBtn	Button to toggle between light and dark themes.
clock	Displays the current time, updated every second.
calcBtn	Calculates the percentage based on input marks from three subjects.
quoteBtn	Button to display a random motivational quote.
studentForm	Form to capture student name, email, and roll number.
form.addEventListener("submit")	Handles form submission and validates input data.

8. TEST CASES AND EXECUTION DATA

The following test cases are based directly on the implemented To-Do List code. They verify task addition, input validation, task display, deletion, and updating of the task list.

Test Case	Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Open webpage	Open the HTML file in a browser	To-Do List webpage should load successfully	Webpage loaded successfully	Pass
TC-02	Add a single task	Enter "Complete assignment" and click Add	Task should be added to the task list	Task displayed successfully	Pass
TC-03	Add multiple tasks	Enter three different tasks and click Add after each	All entered tasks should be displayed in the list	All tasks displayed successfully	Pass
TC-04	Empty input validation	Click Add without entering a task	Alert "Please enter a task!" should be displayed	Alert displayed correctly	Pass
TC-05	Clear input after adding	Enter a task and click Add	Input field should be cleared	Input field cleared successfully	Pass
TC-06	Delete a task	Add a task and click its Delete button	Selected task should be removed from the list	Task deleted successfully	Pass
TC-07	Delete one task from multiple tasks	Add three tasks and delete the second task	Only the selected task should be removed	Selected task removed successfully	Pass
TC-08	Delete all tasks	Add multiple tasks and delete them one by one	Task list should become empty	All tasks deleted successfully	Pass
TC-09	Add task after deletion	Add a task, delete it, and add another task	New task should be added and displayed correctly	New task displayed successfully	Pass

9. OUTPUT



10. RESULT

Thus, a **dynamic webpage** was successfully developed using **HTML, CSS, and JavaScript**. The application allows users to dynamically add tasks, mark tasks as completed, and delete tasks. JavaScript DOM manipulation and functions were successfully implemented, and the application was verified using multiple test cases.